# Trustworthy AI Assignment

## Task: Implementing Input and Output Guardrails

### Objective

Design and implement a guardrail system for an AI application that processes user inputs and generates responses. Your system should demonstrate how input and output guardrails can improve privacy, safety, and reliability. (Use the github Repo https://github.com/aashishpokharel/kss_trustworthy_ai for references)

The assignment focuses on implementing practical controls, testing their effectiveness, and understanding their limitations. You may choose the application domain, provided your choice allows you to demonstrate the required guardrails.

### Core Functionality

#### 1. Input Guardrails

Implement the following controls:

- **PII Detection:** Detect and handle personally identifiable information, such as email addresses, telephone numbers, or identification numbers.
- **Prompt-Injection Detection:** Identify attempts to override instructions, bypass restrictions, or extract confidential information.
- **Sensitive-Topic Classification:** Identify requests that require special handling and define appropriate actions for each category.

For each guardrail, document the detection approach, the action taken when a request is flagged, and the limitations of the implementation.

#### 2. Output Guardrails

Implement the following controls:

- **Schema Validation:** Ensure that generated responses conform to a predefined structure and contain valid field types and values.

- **Groundedness Validation:** Evaluate whether generated claims are supported by the reference information provided to the AI application.
- **Sensitive-Information Detection:** Check generated responses for PII or other information that should not be disclosed.

Define how your system handles invalid responses. Possible actions include redaction, regeneration, refusal, or returning a safe fallback.

**3. Testing and Evaluation**

Create a test suite that evaluates the behavior of each implemented guardrail.

Your test suite must contain at least **20 test cases**, covering normal inputs, malicious inputs, sensitive information, invalid output structures, and unsupported claims.

For each test case, record:

- Test input and relevant reference information.
- Expected behavior.
- Actual behavior.
- Pass/fail status.

Report appropriate evaluation metrics, including precision and recall where ground-truth labels are available. Also report false positives and false negatives.

**4. Failure Analysis**

Identify at least two failure cases in your implementation. These may include missed prompt injections, undetected PII, incorrectly rejected legitimate requests, or unsupported claims passing the output validator.

For each case, explain why the guardrail failed, what the consequences could be, and how you would improve it.

**5. Guardrail Trade-offs**

Evaluate how your implementation balances safety and usability.

Discuss at least one case where stricter controls improve safety but reduce usability, and one case where a less restrictive approach could expose the system to additional risk.

# Report

Submit a short report addressing the following:

**a. Design and Architecture**

Explain the purpose of each guardrail, how the components interact, and why the controls are applied at their selected stages.

**b. Implementation Choices**

Explain the detection and validation techniques used, why they were selected, and what limitations they have.

**c. Evaluation Results**

Present the test results and metrics. Identify which guardrails performed well, which failed, and what the results do not establish.

**d. Risk Assessment**

Identify the most important remaining risks and state whether your implementation would be suitable for deployment, suitable only with restrictions, or not ready for deployment.

## Deliverables

1. **Source code:** A runnable implementation of the input and output guardrails.
2. **Test suite:** At least 20 test cases with expected and actual outcomes.
3. **Evaluation report:** Metrics, failure analysis, trade-offs, and remaining risks.
4. **Architecture diagram:** A diagram showing the guardrail components and their interactions.
5. **README:** Setup instructions, usage examples, design decisions, and evaluation results.